Story: The LLM Application over 40+ Manufacturing Systems

STAR format, about 2 minutes spoken. The production-ML version of the Moderna log-mining story.

Situation

"Moderna runs manufacturing on more than forty software systems - some built in-house, some bought off the shelf, stitched together. Nobody could answer a simple question: which features are people actually using, and which licenses are we paying for that nobody touches."

Task

"My team built an LLM application that answers those questions directly from system logs - ask it what users keep asking for, and it tells you, grounded in the telemetry."

Action

"The hard part was never the model call. It was everything around it. The logs were unstructured - JSON blobs, sometimes literal PDFs - so we built a normalization layer over S3 and Redshift first. Then the grounding problem: the model had to answer from our data, not from its memory. Then the guardrail problem: we had no clean way to separate PII and restricted data from everything else, so we had to solve what the model is allowed to see before we could ship anything. We also learned about coupling the hard way - when OpenAI changed their API format between model versions, our tightly-coupled code broke, and we rebuilt around model-agnostic interfaces."

Result

"The application surfaces the features users ask about most, which let us cut usage-based license fees and retire redundant tools - inside our existing AWS footprint, no new systems, no new spend."

Where this story plays

- Production LLM applications: grounding, guardrails, shipping inside constraints
- ML systems design: interface coupling, model-agnostic architecture
- "Tell me about a technical decision you would revisit" - the OpenAI coupling lesson
- Cost-aware engineering leadership

Follow-ups, with the honest answers

- "How did you evaluate answer quality?" - "Honestly, this is the gap. We validated against known usage patterns in the dashboards rather than a formal eval harness. If I rebuilt it, a golden question set with human-graded answers would come before any demo."
- "Why not buy an off-the-shelf observability tool?" - "Two reasons: months of security approvals for any new system, and none of them answered the actual question, which was what users keep asking for across forty systems."
- "What broke when the model changed?" - "Tightly-coupled code against OpenAI's request format. The fix was model-agnostic interfaces - never let one vendor's schema shape your internals."